Project - Submission Sheet

Software Testing - Random Testing

Names: Kavya Dave
Group: Group 4

Software Testing - Summer 2024 - May 7, 2025 - Sibylle Schupp / Daniel Rashedi

Task 1 - Random Testing

a) Add the 4 generated Python tests of each group member to your repository in the corresponding folder. Add a README.md file that explains how to build / run the tests.

b) Provide screenshots of your terminals that indicate your execution of the new tests:

Answer:

Screenshots of test execution:

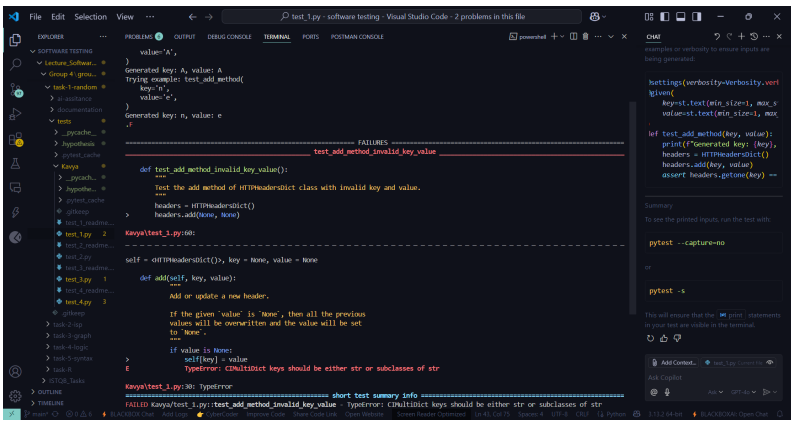

Figure 1: Execution of Test 1

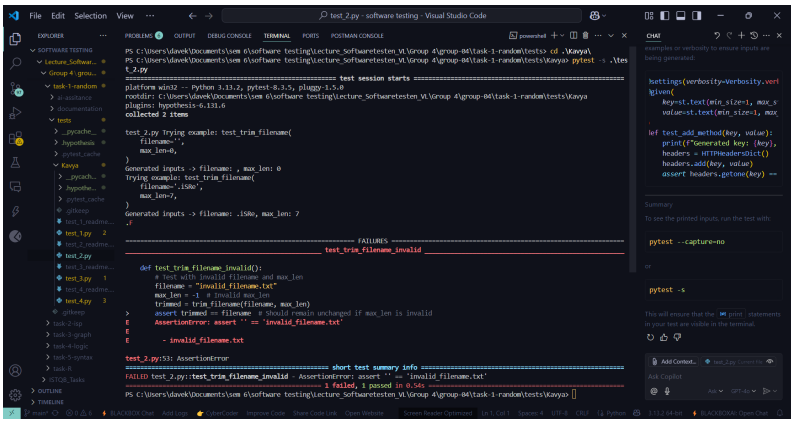

Figure 2: Execution of Test 2

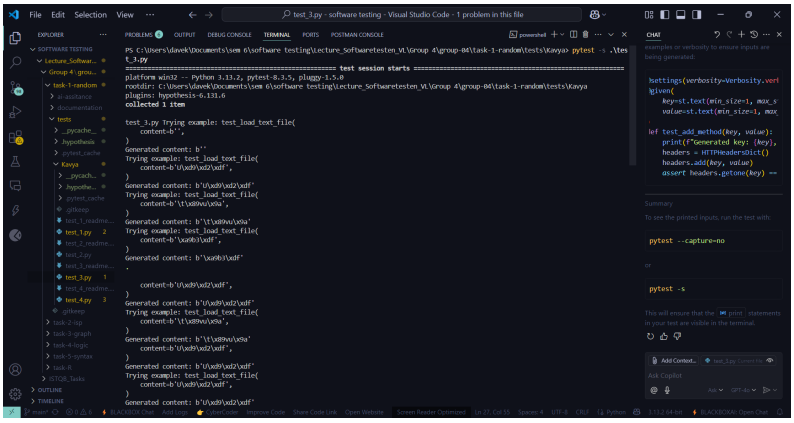

Figure 3: Execution of Test 3

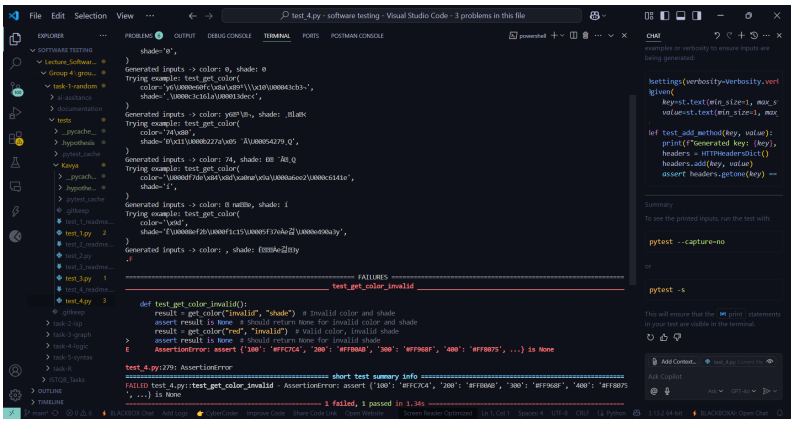

Figure 4: Execution of Test 4

c) Assess the input values generated by `hypothesis`. Do the tests represent meaningful scenarios?

Answer:

Test	Strategy	Realistic	Edge Cases	Notes
`test_add_method`	`st.text(1,10)`	Yes	Yes	Simulates header fields, varyin length and duplicates
`test_trim_filename`	`text + int(0,300)`	Yes	Yes	Tests truncation, extensions, short /long cases
`test_load_text_file`	`st.binary(0,100)`	Yes	Yes	Covers empty, corrupt, and valid fil bytes
`test_get_color`	`st.text(1,10)`	Partially	Yes	Unusual/unknown color names empty input

Table 1: Assessment of Hypothesis-Generated Inputs

AI Review Task

Assess the correctness and understandability of the LLM's generated tests applying the

paper's methodology.

Answer:

Group: Kavya LLM
Prompt Summary: $Hypothesis - basedtestfor`trim_filename`.$
Generated Test Code:

```
@settings(verbosity=Verbosity.verbose, max_examples=2)
@given(
    filename=st.text(
        min_size=0,
        max_size=10,
        alphabet=string.ascii_letters + string.digits + ".-_"
    ),
    max_len=st.integers(min_value=0, max_value=10)
)

def test_trim_filename(filename, max_len):
    trimmed = trim_filename(filename, max_len)

    print(f"Generated inputs -> filename: {filename}, max_len: {max_len}")

    # 1. The trimmed filename should not be longer than max_len
    assert len(trimmed) <= max(max_len, len(os.path.splitext(filename)[1]))

    # 2. If the filename was already short enough, it should remain the same
    if len(filename) <= max_len:
        assert trimmed == filename

    # 3. The trimmed filename should still contain only characters from the original
    assert set(trimmed).issubset(set(filename))

    # 4. If the original had an extension, the trimmed filename should still end with that extension (unless fully trimmed)
    name, ext = os.path.splitext(filename)
    if ext and len(name) > 0 and len(filename) > max_len:
        # If possible, the extension should be preserved
        assert trimmed.endswith(ext) or len(trimmed) < len(ext)

def test_trim_filename_invalid():
    # Test with invalid filename and max_len
    filename = "invalid_filename.txt"
    max_len = -1  # Invalid max_len
    trimmed = trim_filename(filename, max_len)
    assert trimmed == filename  # Should remain unchanged if max_len is invalid

    # Test with empty filename and valid max_len
    filename = ""
```

```
    max_len = 5
    trimmed = trim_filename(filename, max_len)
    assert trimmed == filename  # Should remain unchanged if filename is empty
```

Assessment:

- **Correctness:** Assertions clearly enforce constraints: max length, content integrity, extension preservation.
- **Coverage:** Strong coverage of boundary conditions, including zero-length, edge of truncation and invalid cases.
- **Understandability:** Well-structured and commented, readable even by non-authors.
- **Improvement Suggestion:** Consider adding checks for filenames with only extensions (e.g., '.txt') or names that are entirely trimmed.